

Proposal Defense Reviewer

TrikeKoTo: A Tricycle Ride-Hailing and Dispatch System for TODA Chapters
Jelo Jian Sabroso | PRMSU - San Marcelino | September 2026

How to use: read each question, answer it aloud in your own words, then check against the answer. Don't memorise word for word; panels notice.

A. Problem and relevance

Q1. Why is this study needed? Grab and Move It already exist.

They don't serve TODA tricycles, or they bypass the chapter. TrikeKoTo keeps the TODA in control: the chapter approves who drives, fares follow the posted tariff, and officers can suspend drivers or stop all bookings.

Q2. Is this a real problem in San Marcelino?

Answer from your data gathering: interviews, surveys or observation at the terminal. If none yet: "We will validate it with the chapter officers and commuters during data gathering." Never invent numbers.

Q3. What is new about your study?

Three things together:

- Ride-hailing built around a TODA chapter's rules.
- Server-side nearest-driver dispatch that hides drivers' locations from commuters.
- ID verification designed for the Data Privacy Act.

Q4. Who benefits?

- **Commuters:** less waiting, and they know who is coming.
- **Drivers:** fewer empty trips.
- **The chapter:** control over drivers and a record of rides.
- **Researchers:** a model for local transport groups.

B. Features and design

Q5. Why doesn't the app show or charge a fare?

TODA fares are set by municipal ordinance and posted at the terminal. A second price on a phone could only contradict the official one, and the dispute would land on the driver. Fares are paid in cash.

Q6. Why must commuters verify an ID? Isn't that a barrier?

It protects drivers from fake bookings and scams; a prepaid SIM alone is too cheap to stop trolls. It is done once per account. The trade-off is slower sign-up, which user testing will measure.

Q7. What if a driver has no smartphone?

They can't take app bookings but still pick up passengers the normal way. The app adds a channel; it doesn't replace the terminal.

Q8. What if the commuter or driver has no internet?

Booking needs mobile data; that is a stated limitation. Map tiles are cached on the phone, and the server keeps searching even if the commuter closes the app.

Q9. Why Filipino and English?

Many drivers are more comfortable in Filipino, while officers and the panel may prefer English. Users can switch anytime; Filipino is the default.

Q10. What if the driver cancels or never shows up?

The driver can cancel, which ends the ride, and the commuter books again. Commuters can rate the driver and report problems to the chapter through the app's feedback.

Q11. Is there an emergency or safety feature?

The commuter sees the driver's name and plate, can call them, and can track the tricycle live. A dedicated emergency button is not in scope; it is a good recommendation for future work.

C. Technical

Q12. Explain your dispatch algorithm.

A greedy nearest-first search. The server shortlists approved, online drivers within 5 km by straight-line distance, then ranks them by road distance. The nearest driver gets 15 seconds to accept; if not, the next is asked, up to 10 drivers or 5 minutes.

Q13. Why greedy? It isn't globally optimal.

It suits one booking at a time at chapter scale, and it is simple, fast and explainable. Optimal assignment matters when many rides compete for many drivers at once, which one TODA chapter rarely has.

Q14. What if two drivers accept at the same time?

A database transaction plus a security rule means the ride can only be claimed while it is unassigned. Exactly one driver wins; this is tested with simultaneous accepts.

Q15. Why Flutter and Firebase?

- **Flutter:** one codebase for Android and web.
- **Firebase:** managed sign-in, real-time database, notifications and server functions, with no servers to maintain.
- **Cost:** near zero at chapter scale.

Q16. How do you prevent a fake GPS location?

Honestly, a rooted phone can fake GPS. But only chapter-approved, ID-verified drivers can go online, so the person faking is identifiable and can be suspended: accountability rather than technical prevention.

Q17. What if Firebase goes down?

The app can't book during an outage. Google's managed services are more reliable than a server we would run ourselves; it is an accepted dependency.

Q18. Can it scale?

Firebase scales automatically. The practical limit is the free public map-routing services, which would be replaced by a self-hosted routing server for heavier use.

Q19. How much does it cost to run?

At chapter scale, within Firebase's free usage allowances. A PHP 500 monthly budget alert is set as a safeguard.

Q20. How did you test it?

- Unit and widget tests for the app.
- 216 security-rule tests, including attacks: reading others' data, self-approval, hijacking a ride.
- A concurrency test for double-accepts.
- Manual end-to-end runs on real phones, plus a security review.

D. Security and data privacy

Q21. How do you comply with RA 10173 (Data Privacy Act of 2012)?

- **Access:** only chapter officers can see IDs.
- **Consent:** shown in full and recorded with a timestamp.
- **Retention:** ID photos and numbers are deleted automatically after 90 days.
- **Withdrawal:** users can delete their ID; commuters, their whole account.
- **Ethics:** real IDs won't be collected until the ethics review is complete.

Q22. If the ID is deleted after 90 days, how do you know the user was verified?

Only a record that verification happened is kept, with no ID number or photo. The ID itself is deleted without asking the user to verify again.

Q23. Where is the data stored, and who can access it?

In Google Firebase. Security rules on the server decide every read and write, so the app is never trusted. Commuters can't see other commuters' rides; drivers can't see other drivers.

Q24. Can someone make a fake app to access your data?

App Check is enforced, so only the genuine app reaches the database and the dispatch function. Scripts and modified apps are refused.

E. Methodology and evaluation

Q25. What development methodology did you use?

Name the model in your Chapter 3, then: *"We built it iteratively in phases: requirements, backend and security, features, maps and notifications, testing, deployment."*

Q26. Why only 5 drivers and 5 commuters?

Usability research shows about five users per group uncover most usability problems. The SUS score is reported with its sample size and not over-generalised; the pilot adds real-use data.

Q27. What is SUS, and how is it scored?

The System Usability Scale: 10 statements rated 1 to 5, converted to a 0-100 score. About 68 is average; above 80 is good.

Q28. How will you measure success?

- Task completion rate and time.
- SUS score.
- Whether drivers say they would use it for real work: the real acceptance test.
- In the pilot: rides completed, and the share of bookings that found no driver.

Q29. Is the SUS valid in Filipino?

Our Filipino version is a working translation, not a validated instrument, and that is stated as a limitation.

F. Feasibility and sustainability

Q30. Has the TODA agreed to this?

Answer truthfully. If not yet: *"The next step is to ask the chapter officers' approval before approaching drivers."*

Q31. Do you need permits from the LGU or LTFRB?

The app doesn't change franchises, routes or fares; it only connects commuters to already-franchised TODA members. We will consult the chapter and the local transport office before a pilot.

Q32. Who maintains it after you graduate?

There is an operations runbook and a user manual. The chapter officer handles daily administration in the app; technical maintenance needs a designated person or the university.

Q33. What if drivers don't adopt it?

That is the key risk, which is why user testing asks drivers directly and the pilot starts small, with a stop switch.

G. Tough or trap questions

Q34. Is the system finished?

"The prototype is built, deployed and tested. What remains is testing with actual chapter members, a native language review, ethics clearance and the pilot." Never say "ready for the public."

Q35. Did you build this yourself?

Be honest about your process and tools. Know every part well enough to explain it: dispatch, security rules and data flow.

Q36. What is the weakest part of your system?

"It depends on drivers being online nearby and on mobile data. It is only useful once the chapter adopts it, so adoption is our biggest risk." Admitting a weakness scores better than claiming none.

Q37. Why should this be approved?

"It solves a local problem, respects the existing TODA structure and the law, the prototype already works, and the evaluation plan is realistic."

Numbers to know by heart

Search radius	5 km (set by the chapter)
Time to accept an offer	15 seconds per driver
Drivers tried per booking	Up to 10
Search gives up after	5 minutes
Double-accept	Exactly 1 winner (transaction + rule)
Security-rule tests	216, including attack scenarios
ID photo and number kept for	90 days, then deleted automatically
Build progress	89 of 93 steps
SUS benchmarks	68 = average, above 80 = good
UAT participants	5 drivers + 5 commuters
Budget alert	PHP 500 per month
App version	1.0.5 (6), Android and trikekoto.web.app
Tech stack	Flutter + Firebase; OpenStreetMap, OSRM, Nominatim

If you get stuck

You don't know	"Thank you, that's a good point. I'll look into it and include it in the revision."
You disagree	"I understand the concern. Our reasoning was... but we're open to adjusting."
The question is unclear	"May I clarify: are you asking about...?"

Prepare these yourself

Only you know these answers. Write them down before the defense.

Related studies: 2-3 from your review of related literature, and how TrikeKoTo differs

Why you chose this topic
